Лабораторная работа №8 "Контейнеризация приложения с Docker"

Лабораторная работа №8. Контейнеризация приложения с Docker

Ветка репозитория: `28`

---

1. ЦЕЛЬ РАБОТЫ

Освоение технологии контейнеризации Docker: создание `Dockerfile` для backend и frontend, сборка образов, запуск контейнеров, настройка многоконтейнерного приложения с помощью Docker Compose **для упаковки и развёртывания вашего курсового проекта**.

---

2. НЕОБХОДИМЫЕ ЗНАНИЯ

Для успешного выполнения работы студент должен знать:

- Основы Docker (образ, контейнер, Docker Hub).
- Инструкции Dockerfile: `FROM`, `WORKDIR`, `COPY`, `RUN`, `EXPOSE`, `CMD`.
- Сети и тома в Docker.
- Docker Compose (файл `docker-compose.yml`).
- **Альтернативные реестры образов** (GitLab Container Registry, зеркала) – в связи с ограничениями доступа к Docker Hub в РБ.

  Важное примечание для студентов в Республике Беларусь:
  Docker Hub может быть недоступен без дополнительной настройки.
  Рекомендуется:

- Использовать публичные зеркала (например, `mirror.gcr.io`, `dockerhub.timeweb.cloud`) – настройки указываются в `/etc/docker/daemon.json`.
- Либо использовать GitLab Container Registry (если репозиторий проекта размещён в GitLab) – тогда образы можно загружать и скачивать через `registry.gitlab.com`.
- Для образов `postgres`, `mongodb`, `node`, `nginx` также доступны альтернативные реестры (например, `quay.io`).

---

3. ЗАДАЧИ РАБОТЫ

- Создать `Dockerfile` для Node.js/Express приложения (backend вашего курсового проекта).
- Создать `Dockerfile` для React-приложения (frontend) с многоэтапной сборкой.
- Написать `docker-compose.yml` для запуска backend, frontend и базы данных (PostgreSQL или MongoDB – в зависимости от вашего выбора).
- Собрать образы и запустить всё приложение командой `docker-compose up --build`.
- Проверить работу приложения в контейнерах.
- **Добавить минимум одно из дополнительных требований** (по выбору, в зависимости от специфики вашего проекта).

---

4. ПОРЯДОК ВЫПОЛНЕНИЯ (общий для всех вариантов)

Часть 1. Dockerfile для backend

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
EXPOSE 5000
CMD ["node", "server.js"]
```

Часть 2. Dockerfile для frontend (многоэтапная сборка)

```
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Часть 3. docker-compose.yml (пример для PostgreSQL)

```
version: '3.8'
services:
  backend:
    build: ./backend
    ports:
      - "5000:5000"
    environment:
      - DB_URL=postgresql://user:pass@db:5432/mydb
    depends_on:
      - db
  frontend:
    build: ./frontend
    ports:
      - "80:80"
  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: mydb
    volumes:
      - pgdata:/var/lib/postgresql/data
volumes:
  pgdata:
```

Часть 4. Сборка и запуск

```
docker-compose up --build
```

Часть 5. Проверка

- Открыть `http://localhost` – должна открыться frontend-часть.
- Проверить, что backend доступен по `http://localhost:5000`.
- Просмотреть список контейнеров: `docker ps`.

Часть 6. Дополнительные требования (выберите одно или несколько)

В зависимости от вашего курсового проекта и потребностей, добавьте одну или несколько из следующих возможностей:

- **Переменные окружения** – использовать файл `.env` для строки подключения к БД и других секретов.
- **Healthcheck для backend** – добавить проверку эндпоинта `/health`.
- **Том для логов backend** – создать volume для папки `/logs`.
- **Сервис `pgadmin` / `mongo-express`** – добавить веб-интерфейс для управления БД.
- **Дополнительный сервис (Redis)** – для кэширования, если это уместно.
- **Изоляция сетей** – разделить сервисы на внутреннюю и внешнюю сети.
- **Reverse-proxy (Nginx)** – настроить Nginx для фронтенда и как прокси для бекенда.
- **Healthcheck для Redis или БД** – с условием `service_healthy` в `depends_on`.

---

5. СОДЕРЖАНИЕ ОТЧЕТА

Отчёт должен содержать:

- Титульный лист, цель работы.
- Теоретические основы контейнеризации (кратко).
- Листинги ваших `Dockerfile` и `docker-compose.yml` с пояснениями.

- Скриншот выполнения `docker ps` (работающие контейнеры).
- Скриншот работы приложения в браузере (`http://localhost`).
- Описание выбранного дополнительного требования и его реализации.
- Ответы на контрольные вопросы.
- Выводы.

---

6. ТРЕБОВАНИЯ К СДАЧЕ

- Отчёт (PDF) загружается в СЭО.
- Ссылка на репозиторий GitHub с кодом (ветка **28**).
- **Индивидуальное выполнение. Вариант** **Темы** определяется по **Генератору вариантов**
- Код должен быть загружен в репозиторий с веткой `28`.

---

7. КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Чем отличается Docker-образ от Docker-контейнера?
2. Для чего используется инструкция `COPY --from=builder` в многоэтапной сборке?
3. Как в Docker Compose организовать связь между контейнерами по имени сервиса?
4. Что такое том (volume) и зачем он нужен для базы данных?
5. Как посмотреть логи запущенных контейнеров?
6. Как уменьшить размер Docker-образа? Назовите не менее 3 способов.
7. Какая инструкция Dockerfile задаёт команду по умолчанию при запуске контейнера?
8. В чём разница между `CMD` и `ENTRYPOINT`?
9. Как в Docker Compose задать зависимость запуска контейнеров (чтобы база данных стартовала раньше backend)?

10. Как выполнить команду внутри запущенного контейнера (например, зайти в оболочку)?
11. Как пробросить порт с хоста в контейнер в Docker Compose?
12. Как в Docker Compose использовать файл `.env` для переменных окружения?
13. Что такое `healthcheck` в Docker и зачем он нужен?
14. Как опубликовать собственный образ в альтернативный реестр (например, GitLab Container Registry)?
15. Что произойдёт, если не смонтировать volume для базы данных? Что будет при перезапуске контейнера?
16. Как в Docker Compose масштабировать сервис (например, backend) до нескольких экземпляров? Какие ограничения?

---

8. ПРИМЕЧАНИЯ

- Работа выполняется **индивидуально**.
- **Ограничения Docker Hub в РБ:** студентам следует заранее настроить зеркала (например, добавить в `/etc/docker/daemon.json` строки `"registry-mirrors": ["https://mirror.gcr.io", "https://dockerhub.timeweb.cloud"]`). Альтернативно можно использовать образы из `registry.gitlab.com` или другие доступные реестры.
- Для вариантов с **MongoDB** рекомендуется использовать образ `mongo:6` или `mongo:7`.
- В отчёте необходимо указать, какие меры были предприняты для обеспечения доступа к образам (использованное зеркало или реестр).

---

9. КРИТЕРИИ ОЦЕНКИ

- Наличие корректных `Dockerfile` для backend и frontend с многоэтапной сборкой для React.
- Корректный `docker-compose.yml`, включающий БД и связь между сервисами.
- Использование томов для сохранения данных БД.

- Выполнение дополнительного требования (по выбору студента).
- Возможность запуска приложения командой `docker-compose up --build` без ошибок.
- Полнота отчёта: скриншоты, листинги, ответы на контрольные вопросы.
- Ссылка на репозиторий с веткой `28` и инструкцией по запуску.